

0.91.3. Le titre, puis le jeu — un cadre de murs

X

Le 0.91.2, et des murs tout autour du terrain : un cadre de X. Le A qui y entre revient aussitôt à sa place d'avant.

1. Ce qu'il faut comprendre

C'est le 0.91.2, avec une chose en plus : un cadre de murs. Les murs sont des **lettres**, comme le M du 0.82 ; ici, le **X**. Il fait le tour du terrain : le haut (ligne 0), le bas (ligne 16), la gauche (colonne 0), la droite (colonne 19).

Le dessiner : deux boucles. La première pose les X du haut et du bas, colonne par colonne (0 à 19). La seconde, ceux des côtés, ligne par ligne (1 à 15 : les coins sont déjà posés).

La ligne 17 reste aux nombres : le cadre s'arrête à la ligne 16, SCORE et TEMPS restent en dessous.

Ce qui est nouveau ici : revenir en arrière. `deplace_croix` ne connaît pas les murs : elle fait le pas. Alors on retient la place du A **avant** (`ax`, `ay`), et **après** le pas on regarde : sur le cadre ? On remet le X (le A l'avait recouvert) et le A revient à sa place d'avant. Tout se passe dans la même image : on ne voit pas le A entrer dans le mur.

`||` veut dire « ou » : `x == 0 || x == 19 || y == 0 || y == 16` est vrai dès qu'une des quatre l'est. `y >= 16` (plus grand ou égal) compte aussi la ligne 17 : le A ne descend jamais sur les nombres.

Le cadre est redessiné à chaque partie, dans `nouvellePartie()` : après le titre et après la fin, l'écran a été vidé.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <couleurTexte>` → [0.87.7. La fonction couleurTexte\(\) — la couleur des lettres](#)
- `#include <texteGrand>` → [0.86.3. La fonction texteGrand\(\) — un texte agrandi](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <deplace_croix>` → [0.66.3. La fonction deplace_croix\(\) — suivre la croix, case par case](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2284 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	105 o	← la console, à l'allumage – la mise en route de la console
texte_grand_0()	195 o	← main() – ta fonction
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
deplace_croix()	217 o	← main() – une fonction fournie par le langage
viderEcran()	33 o	← nouvellePartie(), main() – ta fonction
nouvellePartie()	160 o	← main() – ta fonction
main()	307 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
EcrireTexte	10 o	← texte() – écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() – écrit un nombre en chiffres
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	848 o	← CopierTuiles – la police entière, et 9 tuiles dessinées
"SCORE"	5 o	← texte() – un texte gravé
"TEMPS"	5 o	← texte() – un texte gravé
"APPUIE SUR START"	16 o	← texte() – un texte gravé
"FIN"	3 o	← texte() – un texte gravé
"SCORE"	5 o	← texte() – un texte gravé
"START : REJOUER"	15 o	← texte() – un texte gravé
GRAND_TABLE_0_0_0	12 o	← poser() – une table gravée
GRAND_TABLE_0_1_1	12 o	← poser() – une table gravée
GRAND_TABLE_0_2_2	12 o	← poser() – une table gravée
GRAND_TABLE_0_3_3	12 o	← poser() – une table gravée

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 12	223 o sur place, dans texte_grand_0(), deplace_croix(), nouvellePartie(), main() + AdresseCase (10 o), GRAND_TABLE_0_0_0 (12 o), GRAND_TABLE_0_1_1 (12 o), GRAND_TABLE_0_2_2 (12 o), GRAND_TABLE_0_3_3 (12 o)
texte()	× 6	66 o sur place, dans nouvellePartie(), main() + EcrireTexte (10 o)
bouton()	× 6	30 o sur place, dans deplace_croix(), main() + LireManette (37 o)
nombre()	× 3	30 o sur place, dans main() + EcrireNombre (33 o)
effacer()	× 2	29 o sur place, dans deplace_croix(), viderEcran() + AdresseCase (10 o), EffacerCases (12 o)
couleurFond()	× 1	12 o sur place, dans main()
images()	× 2	4 o sur place, dans chaque_minuteur(), deplace_croix()
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

deplace_croix()	× 1	19 o sur place, dans main()
chaque_minuteur()	× 1	10 o sur place, dans main()
viderEcran()	× 2	6 o sur place, dans nouvellePartie(), main()
nouvellePartie()	× 2	6 o sur place, dans main()
texte_grand_0()	× 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <effacer>           // efface des cases, ou tout le fond
#include <poser>              // pose une tuile sur une case du fond
#include <ALPHABET>           // les lettres de la police : ALPHABET[0] est le A
#include <texte>               // écrit un texte à l'écran
#include <couleurTexte>       // choisit la couleur des lettres
#include <texteGrand>          // écrit un texte agrandi, de 1 à 20 fois
#include <bouton>              // lit un bouton de la manette
#include <deplace_croix>      // une tuile qui suit la croix, case par case
#include <nombre>              // écrit un nombre en chiffres
#include <chaque>              // répond 1 toutes les n millisecondes
```

2. UNE VARIABLE — ECRAN

```
uint8_t ecran = 0;          // 0 = le TITRE, 1 = le JEU, 2 = la FIN
```

3. UNE VARIABLE — X

```
uint8_t x = 5;              // le A : sa colonne...
```

4. UNE VARIABLE — Y

```
uint8_t y = 8;              // ...et sa ligne (le 0.81)
```

5. UNE VARIABLE — PX

```
uint8_t px = 15;            // la pièce, le P : sa colonne...
```

6. UNE VARIABLE — PY

```
uint8_t py = 8;             // ...et sa ligne
```

7. UNE VARIABLE — SCORE

```
uint8_t score = 0;          // les pièces ramassées
```

8. UNE VARIABLE — TEMPS

```
uint8_t temps = 30;         // les secondes qui restent
```

9. UNE VARIABLE — AX

```
uint8_t ax = 5;             // NOUVEAU : la place du A AVANT son pas...
```

10. UNE VARIABLE — AY

```
uint8_t ay = 8;             // ...pour l'y remettre s'il entre dans un mur
```

11. UNE FONCTION — VIDERECRAN()

```
void viderEcran() {  
    for (uint8_t l = 0; l < 18; l++) {  
        effacer(0, l, 20);  
    }  
}  
  
// ---- NOUVEAU : une partie qui commence, TOUT au départ ----  
// Depuis le titre, ET depuis la fin (START : rejouer).
```

12. UNE FONCTION — NOUVELLEPARTIE()

```
void nouvellePartie() {
    x = 5;                      // le A revient à sa place
    y = 8;
    px = 15;                    // le P aussi (il était peut-être en 20, ramassé)
    py = 8;
    ax = 5;                     // et sa place d'avant, la même
    ay = 8;
    score = 0;                  // pas encore de pièce
    temps = 30;                 // 30 secondes de nouveau
    ecran = 1;                  // on est sur le JEU
    viderEcran();
    poser(px, py, ALPHABET[15]);

    // ---- NOUVEAU : le cadre de murs, des X tout autour du terrain ----
    //
    //   colonne : 0 1 2 ...      19
    //   ligne 0 : X X X X X X ... X X X   le haut : ligne 0, colonnes 0 à 19
    //   ligne 1 : X                X
    //   ...      X   A           P   X   les côtés : colonnes 0 et 19
    //   ligne 15: X                X
    //   ligne 16: X X X X X X ... X X X   le bas : ligne 16
    //   ligne 17: SCORE 000  TEMPS 030   la ligne 17 reste aux nombres
    //
    // ALPHABET[23] : la 24e lettre (on compte depuis 0), le X.
    for (uint8_t c = 0; c < 20; c++) {
        poser(c, 0, ALPHABET[23]);      // le haut
        poser(c, 16, ALPHABET[23]);     // le bas
    }
    for (uint8_t l = 1; l < 16; l++) {
        poser(0, l, ALPHABET[23]);      // le côté gauche
        poser(19, l, ALPHABET[23]);     // le côté droit
    }

    texte(0, 17, "SCORE");
    texte(11, 17, "TEMPS");
}
```

13. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    couleurTexte(31, 16, 0);
    texteGrand(4, 5, "JEU", 4);
    texte(2, 12, "APPUIE SUR START");

    while (true) {
        image();

        if (ecran == 0 && bouton(START)) {
            nouvellePartie();          // la première partie
        }

        if (ecran == 1) {
            ax = x;                      // NOUVEAU : on retient où est le A..
            ay = y;
            deplace_croix(x, y, ALPHABET[0], 250); // ...il fait peut-être un pas...

            // ...et s'il est arrivé SUR le cadre (colonne 0 ou 19, ligne 0 ou 16) :
            // 1. on remet le X, que le A venait de recouvrir ;
            // 2. le A revient à sa place d'avant, et s'y pose.
            // || veut dire « OU » : une seule des quatre suffit.
            if (x == 0 || x == 19 || y == 0 || y >= 16) {
                poser(x, y, ALPHABET[23]); // le mur revient
                x = ax;                     // le A recule
                y = ay;
                poser(x, y, ALPHABET[0]);
            }
            if (x == px && y == py) { // le A sur le P : ramassée (le 0.81)
                score = score + 1;
                px = 20;              // la pièce part hors de l'écran
            }
            nombre(6, 17, score);
            nombre(17, 17, temps);
        }

        if (ecran == 1 && chaque(1000)) {
            temps = temps - 1;
            if (temps == 0) {
                ecran = 2;
                viderEcran();
                texte(8, 6, "FIN");
                texte(5, 9, "SCORE"); // NOUVEAU : le score de la partie
                nombre(11, 9, score);
                texte(2, 13, "START : REJOUER");
            }
        }

        // ---- NOUVEAU : sur la fin, START relance une partie.
        if (ecran == 2 && bouton(START)) {
            nouvellePartie(); // la même fonction : tout au départ
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



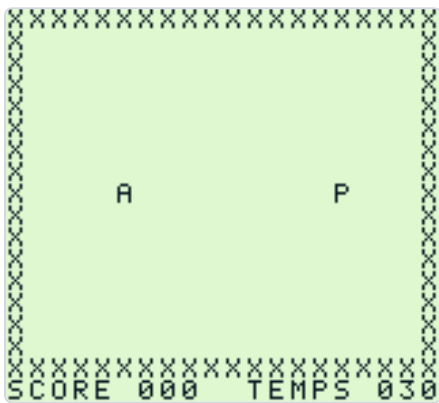
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



START — l'écran a changé

5. Ce qu'on doit voir

START : le terrain entouré de X ; le A se cogne au cadre et ne le traverse pas.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le cadre de X fait le tour du terrain
- ✓ à gauche, le A s'arrête en colonne 1 ($x = 1$)
- ✓ en haut, il s'arrête en ligne 1 ($y = 1$)
- ✓ les murs sont intacts, le A est à sa place

Ensuite : **0.91.4. Le titre, puis le jeu — la pièce revient au hasard** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++